

Documento de Arquitetura de Software

Gnosis

Gestor do Projeto	Gerente do Projeto
Gabriel Willian dos Santos Souza	Lucas Ataide Martins
Gnosissuporte@gmail.com	Gnosissuporte@gmail.com
6199658-9491	6199999999

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
mm/aaaa 24/05/2026	DOC_02	Lucas Ataide	Inserindo o diagrama de Casos de Uso referente as atividades realizadas pelo software.	1.0
05/06/2026	DOC_03	Gabriel Willian	Adicionando diagramas de Sequência	2.0
06/06/2026	DOC_04	Bianca Maria	Adicionando diagramas de classe	2.1
08/06/2026	DOC_04	Marília	Adicionando diagrama de Implantação	2.2
08/06/2026	DOC_04	Lavínia Sobrinho	Adicionando diagrama de pacotes	2.3
09/06/2026	DOC_05	Gabriel Willian	Adicionando itens de qualidade, definições, requisitos e restrições.	2.4
15/06/2026	DOC_05	Gabriel Willian	Adicionando diagrama de Camadas da aplicação	3.0
17/06/2026	DOC_06	Lucas Ataide	Implementando visão lógica e subsistemas	4.0

Sumário

1. INTRODUÇÃO.....	2
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	3
2. REPRESENTAÇÃO ARQUITETURAL.....	4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	5
4. VISÃO DE CASOS DE USO	5
4.1 Casos de Uso significantes para a arquitetura	5
5. VISÃO LÓGICA.....	6
6. VISÃO DE IMPLEMENTAÇÃO.....	8
Caso de Uso [001] & [005].....	8
Caso de Uso [003]	9
6.1.1 Diagrama de Sequência	12
Caso de Uso [001]	13
Caso de Uso [002]	13
Caso de Uso [003]	13
Caso de Uso [004]	14
Caso de Uso [005]	14
Caso de Uso [006]	15
7. VISÃO DE IMPLANTAÇÃO.....	15
8. DIMENSIONAMENTO E PERFORMANCE.....	16
8.1 Volume	16
8.2 Performance.....	16
9. QUALIDADE.....	17

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema Gnosis, usando diversas visões de arquitetura para **representar** diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema.

O documento irá adotar uma estrutura baseada na visão “4+1” de modelo de arquitetura [KRU41].



Figura 1 – Arquitetura 4+1

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao Gnosis, que será desenvolvido pela Bianca Maria, Lucas Ataíde, Lavínia Batista, Gabriel Willian e Marília de Jesus.

1.3 Definições, Acrônimos e Abreviações

QoS – Quality of Service, ou qualidade de serviço. Termo utilizado para descrever um conjunto de qualidades que descrevem as requisitos não-funcionais de um sistema, como performance, disponibilidade e escalabilidade [QOS].

MVC – Model-View-Controller, ou Modelo-Visão-Controlador. Padrão de arquitetura de software que divide a aplicação em três componentes lógicos interconectados: o Modelo (gerenciamento de dados e regras de negócio), a Visão (interface gráfica e apresentação ao usuário) e o Controlador (intermediário que gerencia as requisições e coordena o fluxo entre o Modelo e a Visão) [MVC].

Design Patterns – Padrões de Projeto. Soluções reutilizáveis e consolidadas para problemas comuns que ocorrem durante o desenvolvimento e a modelagem de software, funcionando como guias de boas práticas para estruturar o código de forma limpa, flexível e manutenível [DP].

Supabase – Plataforma de código aberto do tipo Backend-as-a-Service (BaaS), construída sobre o ecossistema do banco de dados relacional PostgreSQL, que fornece de forma integrada ferramentas automáticas como autenticação de usuários, APIs instantâneas, armazenamento de arquivos e sincronização de dados em tempo real [SUPABASE].

1.4 Referências

- [KRU41]: The “4+1” view model of software architecture, Philippe Kruchten, November 1995, <http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>
- [QOS] <https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html>
- [MVC] FOWLER, Martin. Padrões de Arquitetura de Aplicações Corporativas. Porto Alegre: Bookman, 2006.
- [DP] GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos. Porto Alegre: Bookman, 2000.
- [SUPABASE] SUPABASE. Supabase Documentation. Disponível em: <https://supabase.com/docs>. Acesso em: 9 jun. 2026.

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	
Processo	Integradores	Performance, Escalabilidade, Concorrência	
Implementação	Programadores	Componentes de Software	
Implantação	Gerência de Configuração	Nodos físicos	
Caso de Uso	Todos	Requisitos funcionais	
Dados	Especialistas em dados Administradores de dados	Persistência de dados	

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descrever os requisitos de software e restrições que tem um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	JavaScript (utilizando Node.js no ambiente de execução do servidor).
Plataforma	Servidor Docker, utilizando a containerização para empacotar a aplicação e todas as suas dependências operacionais.
Segurança	A autenticação utiliza Supabase Auth com tokens JWT. A API valida o usuário autenticado nas rotas protegidas, enquanto a extensão Chrome utiliza credenciais da sessão para consultar tarefas pendentes e disparar notificações locais.
Persistência	Supabase, utilizando seu banco de dados relacional PostgreSQL subjacente, integrado com políticas de segurança na camada de dados (RLS - <i>Row Level Security</i>).
Padrão Arquitetural	MVC (Model-View-Controller)
Interface / Apresentação	Extensão de Navegador (Chrome Extension).
Comunicação / Integração	API RESTful com tráfego de dados em formato JSON.

Tabela 2 – Exemplo de requisitos e restrições

4. VISÃO DE CASOS DE USO

Lista de casos de uso do sistema:

- **Caso de Uso 01 – Gerenciamento de atividade**
- **Caso de Uso 02 – Gerenciamento de matérias**
- **Caso de Uso 03 – Gerenciamento de conta**
- **Caso de Uso 04 – Visualizar Alertas de Prazos**
- **Caso de Uso 05 – Alteração de status de atividade**
- **Caso de Uso 06 – Sistema de cadastro**

4.1 Casos de Uso significantes para a arquitetura

- **Caso de Uso Significante 01 – Monitorar prazos automaticamente**
- **Caso de Uso Significante 02 – Definir e alterar status de atividades**
- **Caso de Uso Significante 03 – Visualizar Alertas de Prazos**

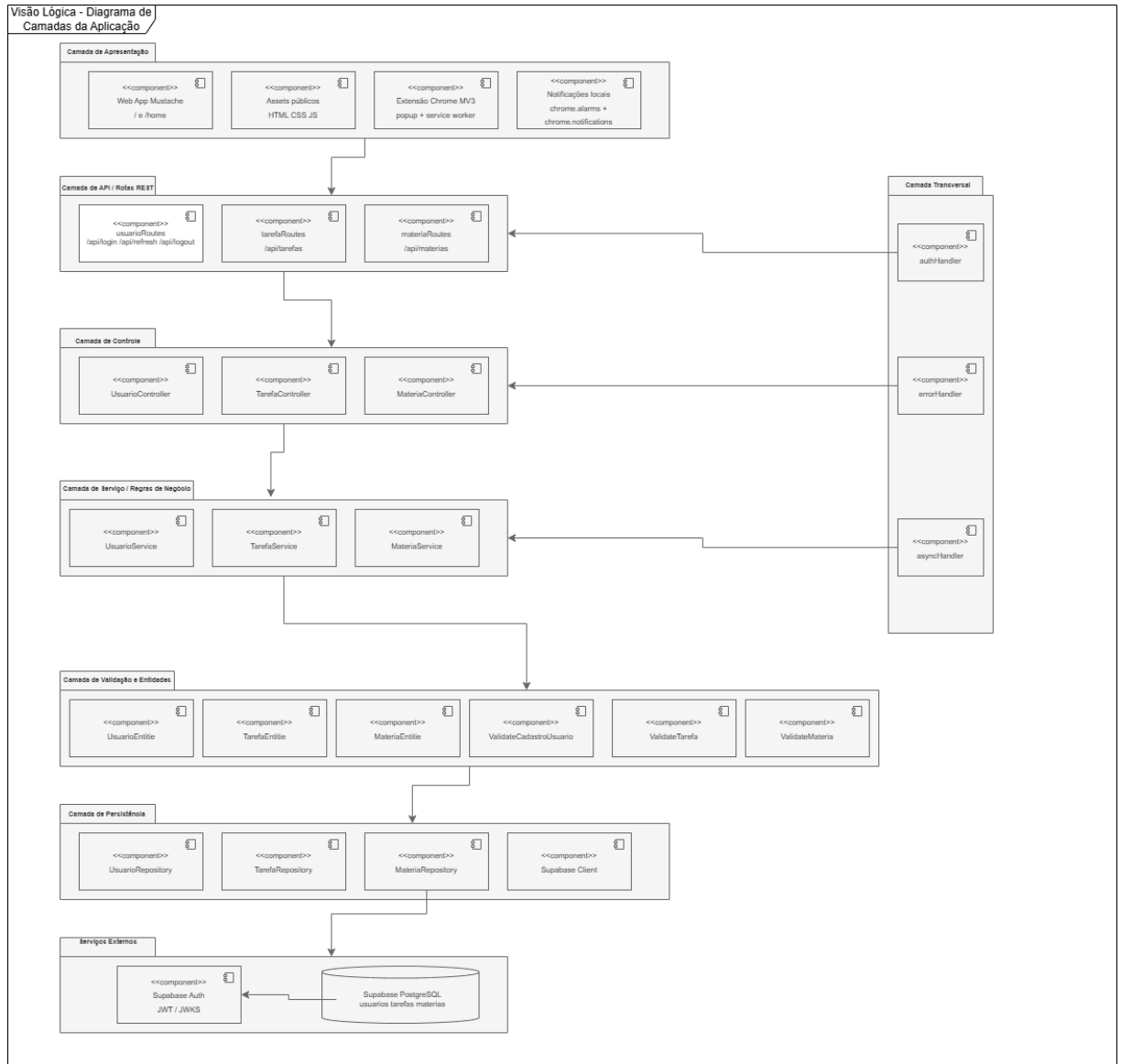


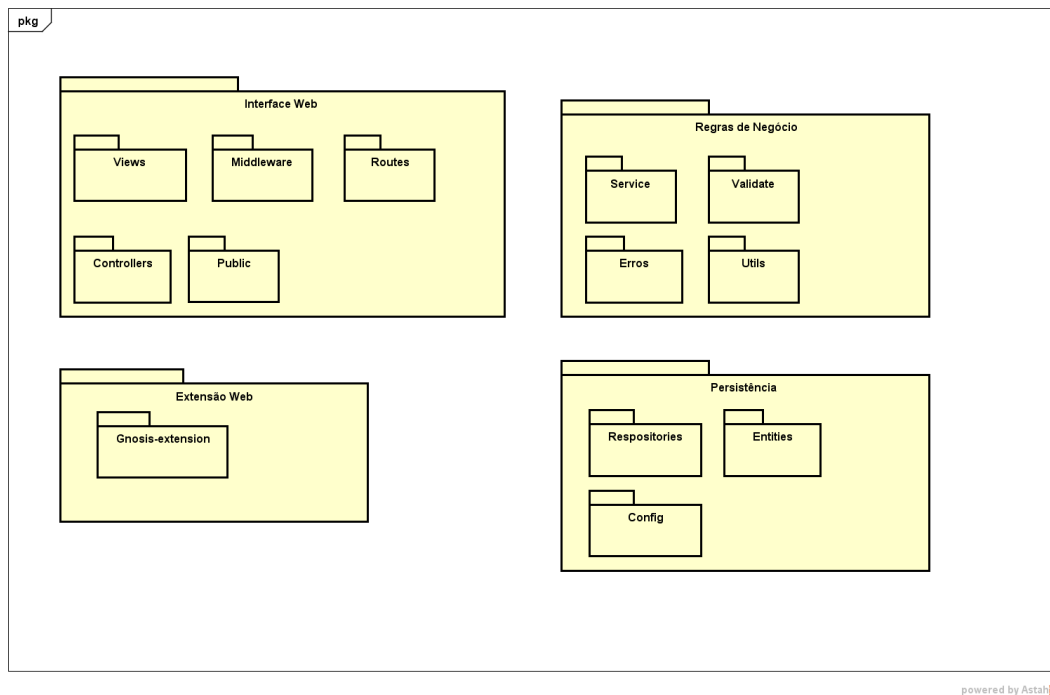
Figura 2 – Exemplo de Diagrama com os casos de uso significativos e atores

5. VISÃO LÓGICA

O sistema Gnosis foi desenvolvido seguindo uma arquitetura em camadas, separando responsabilidades entre apresentação, regras de negócio, persistência de dados e integração com serviços externos, como banco de dados e extensão chrome. Essa abordagem facilita a manutenção, escalabilidade e reutilização dos componentes.

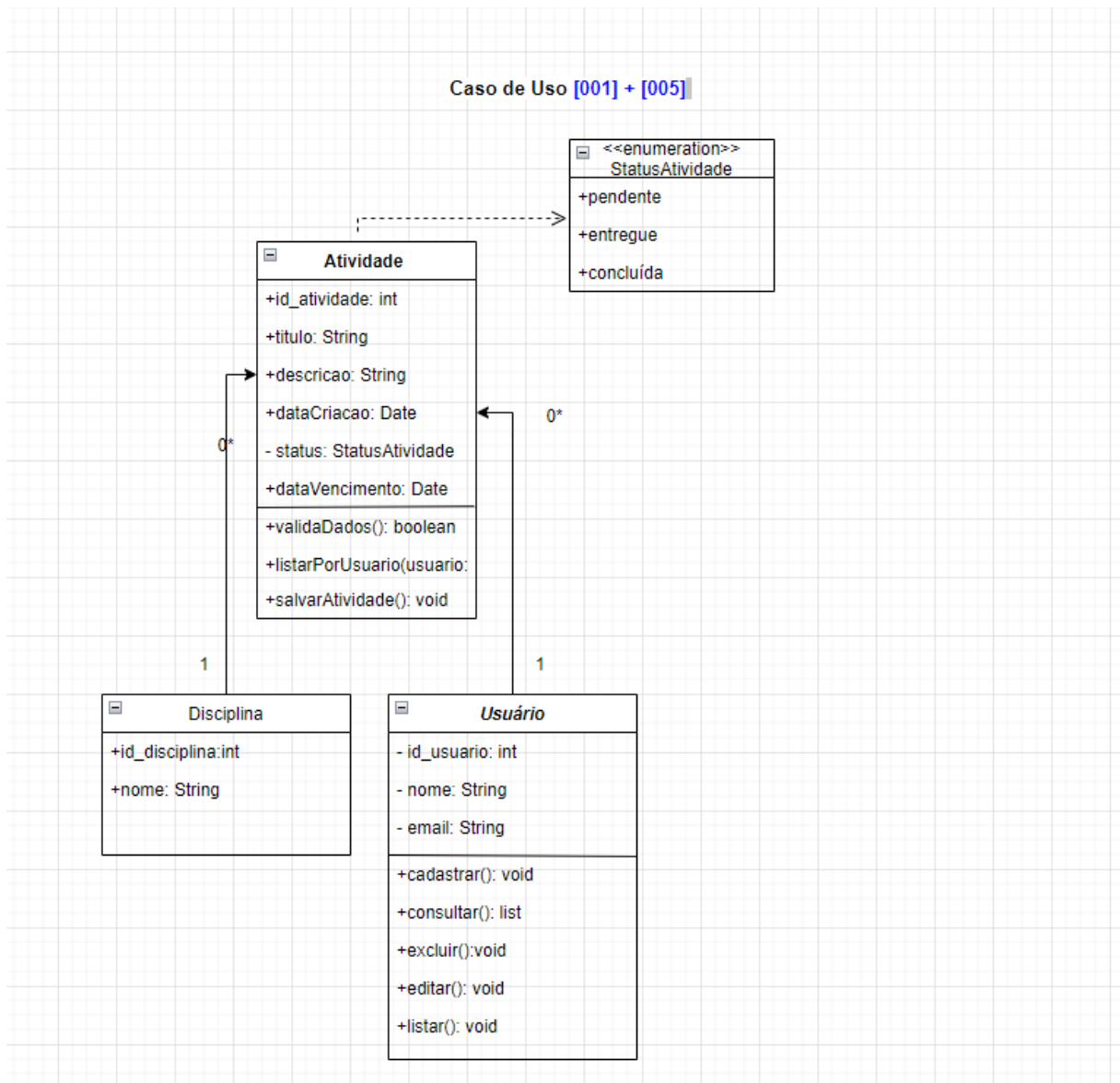
Diagrama de Camadas da Aplicação



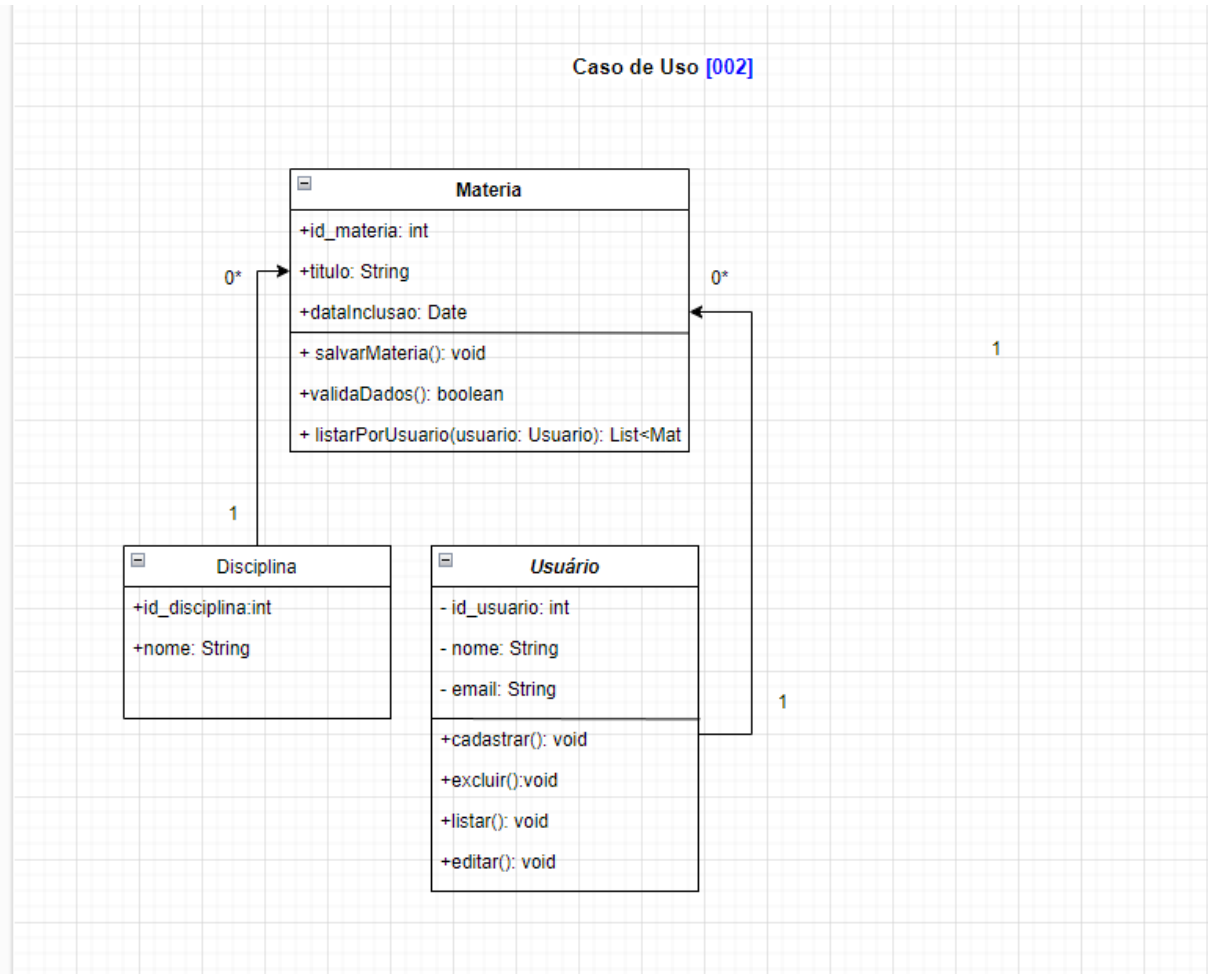


6. VISÃO DE IMPLEMENTAÇÃO

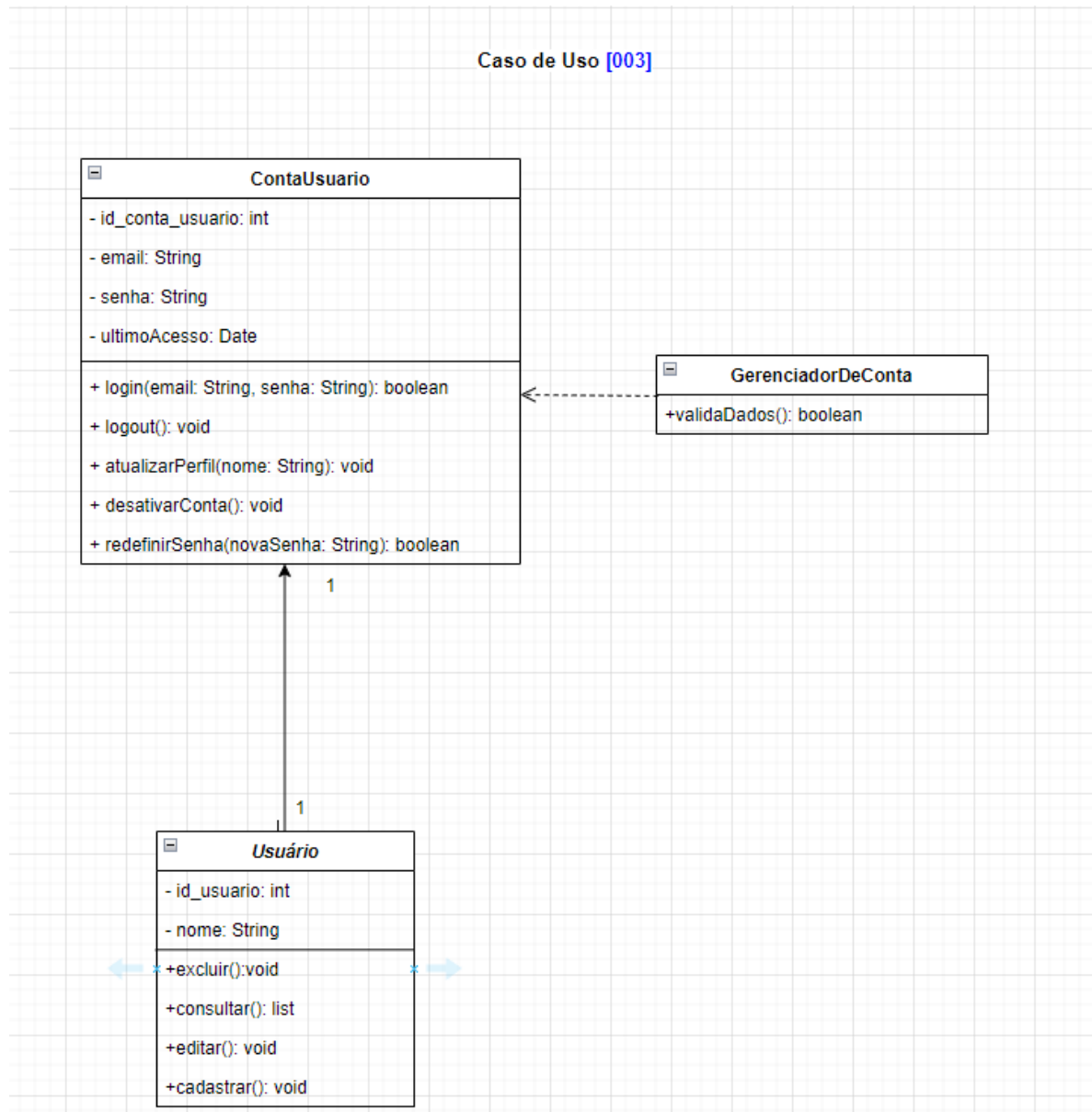
Caso de Uso [001] & [005]



Caso de Uso [002]

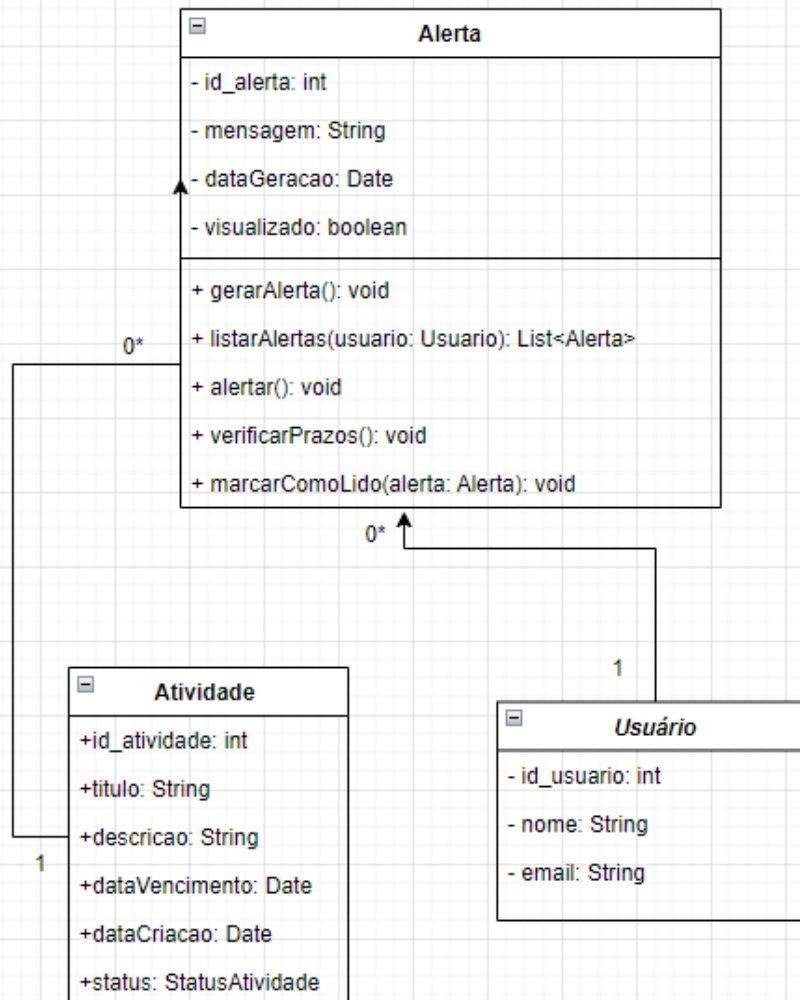


Caso de Uso [003]



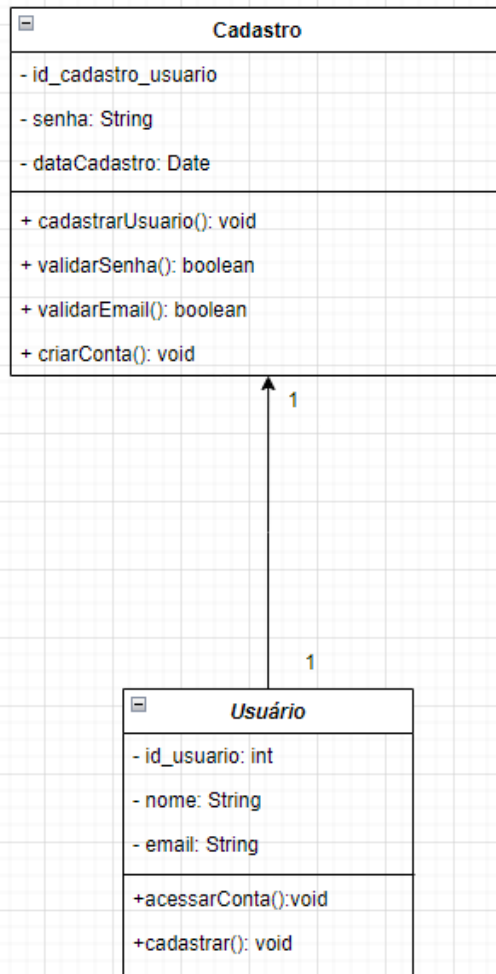
Caso de Uso [004]

Caso de Uso [004]



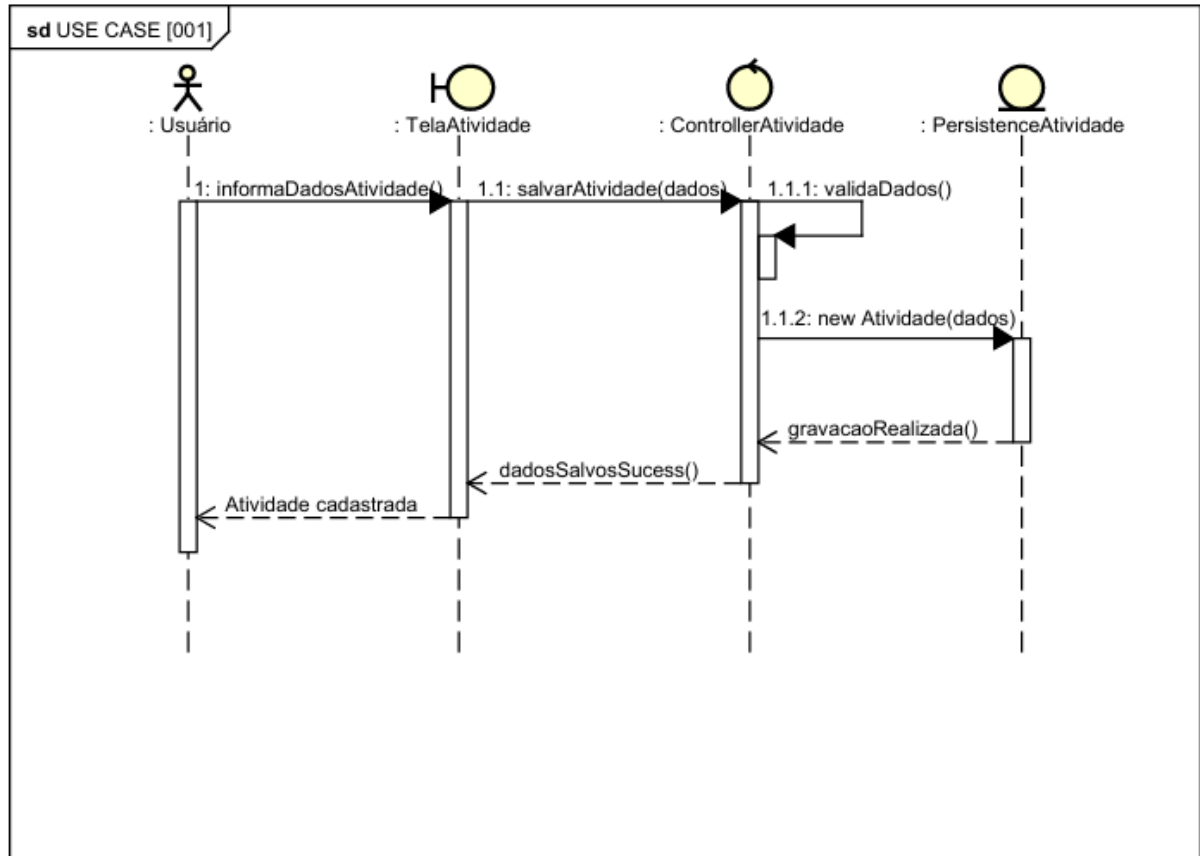
Caso de Uso [006]

Caso de Uso [006]

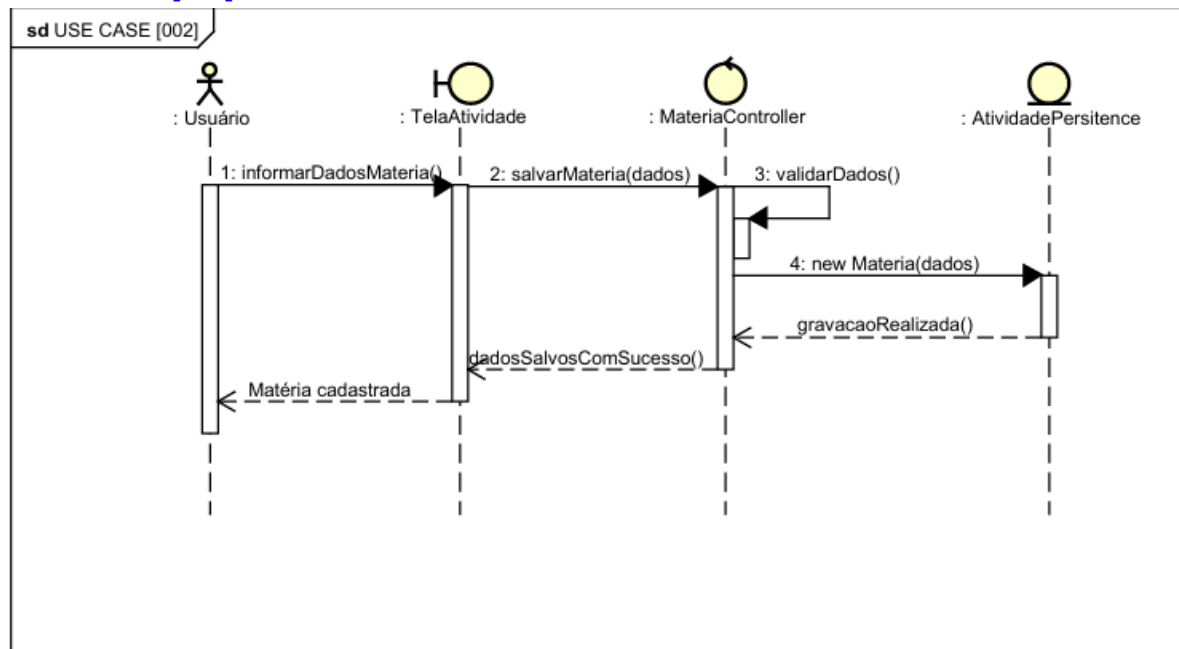


6.1.1 Diagrama de Sequência

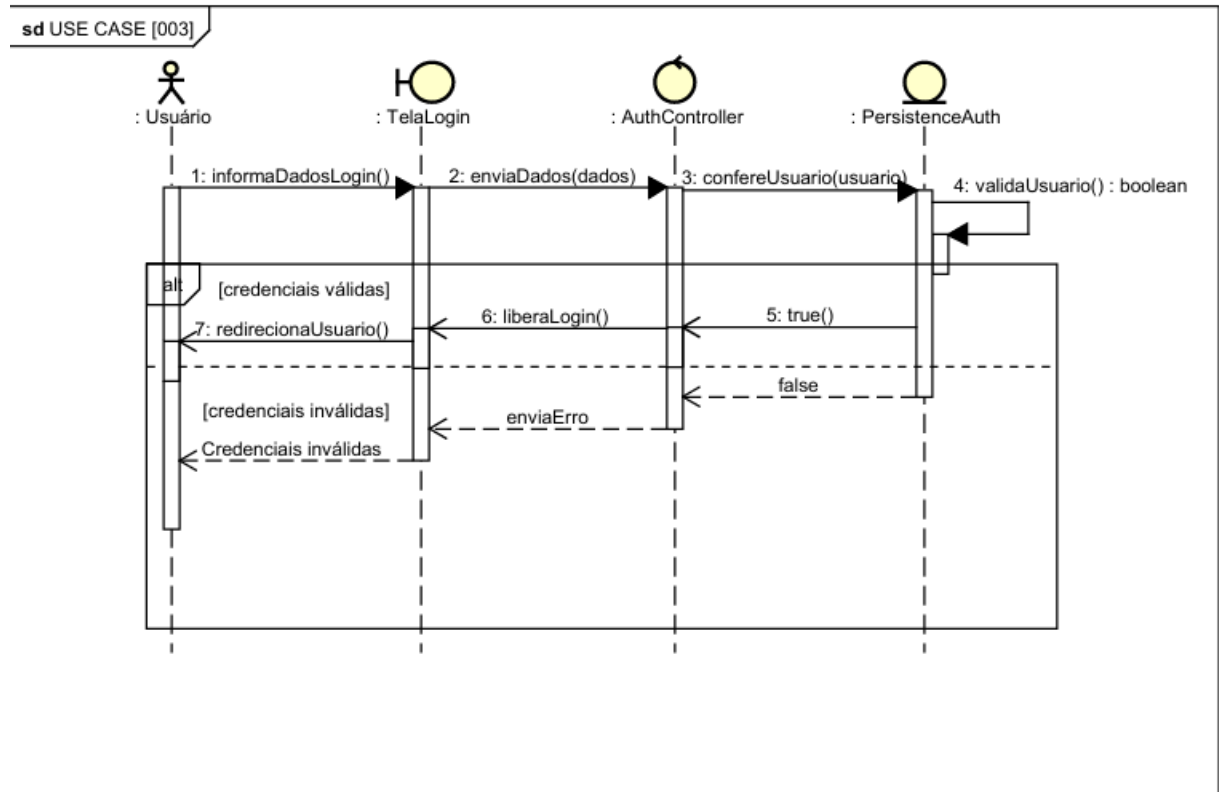
Caso de Uso [001]



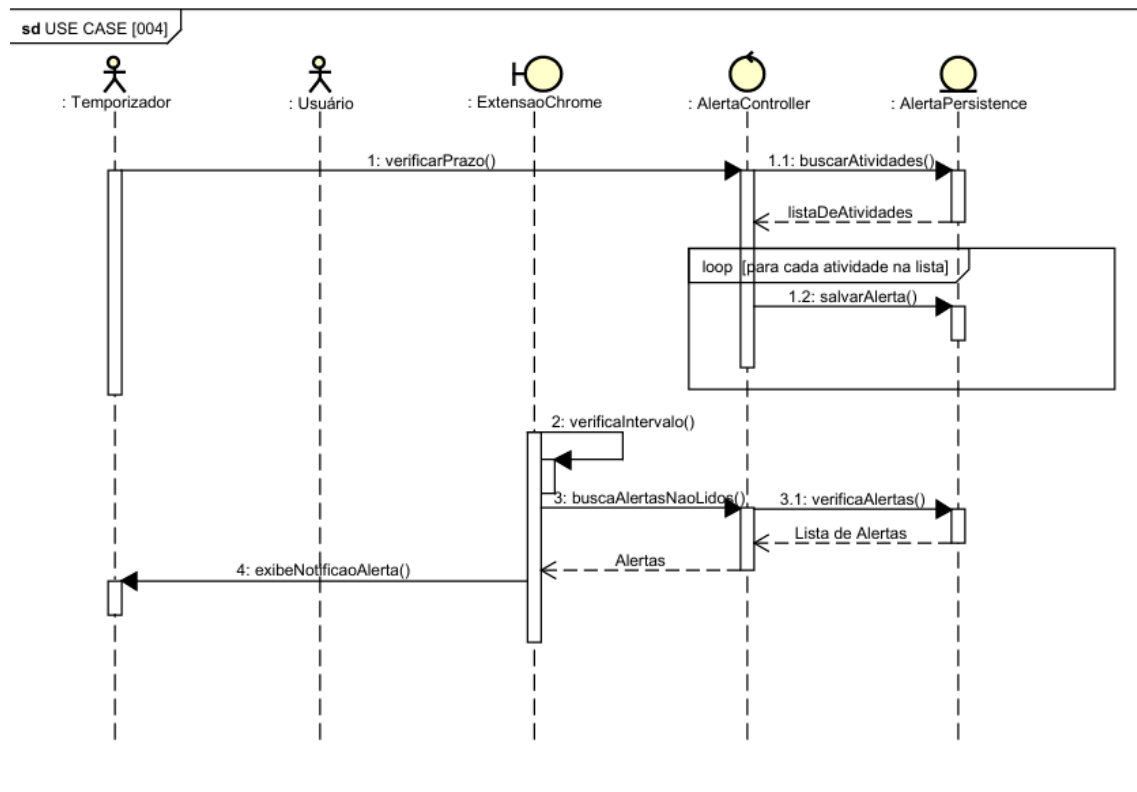
Caso de Uso [002]



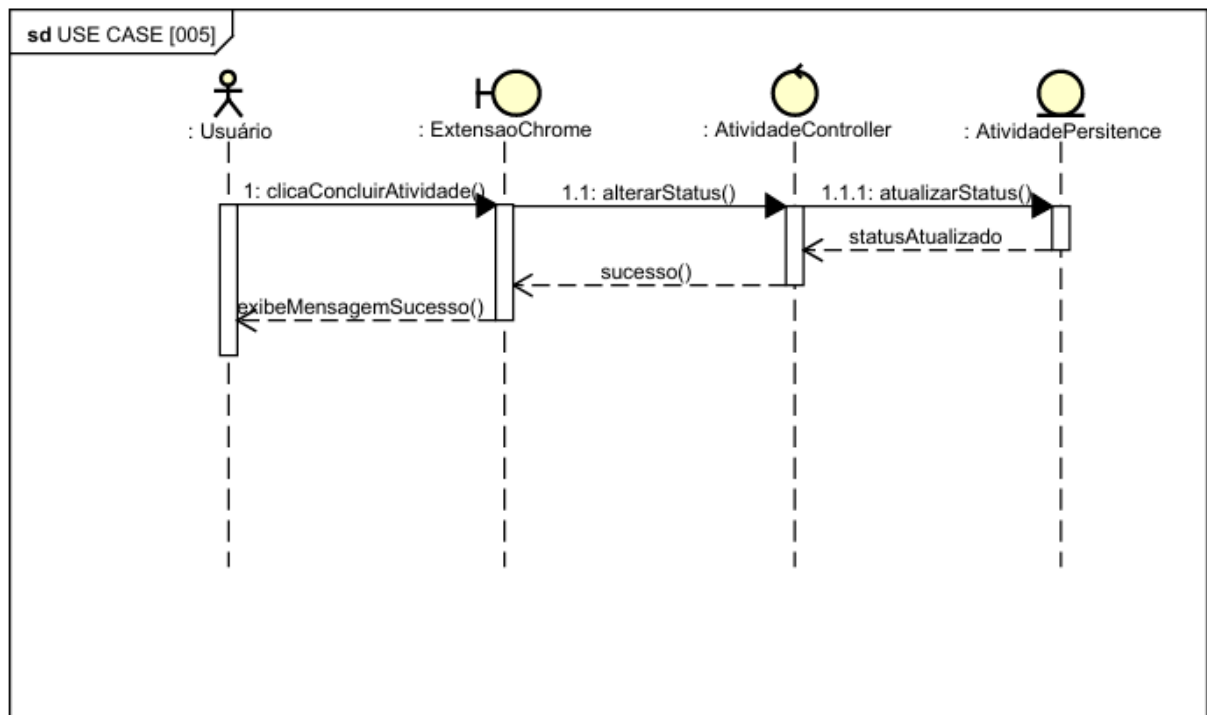
Caso de Uso [003]



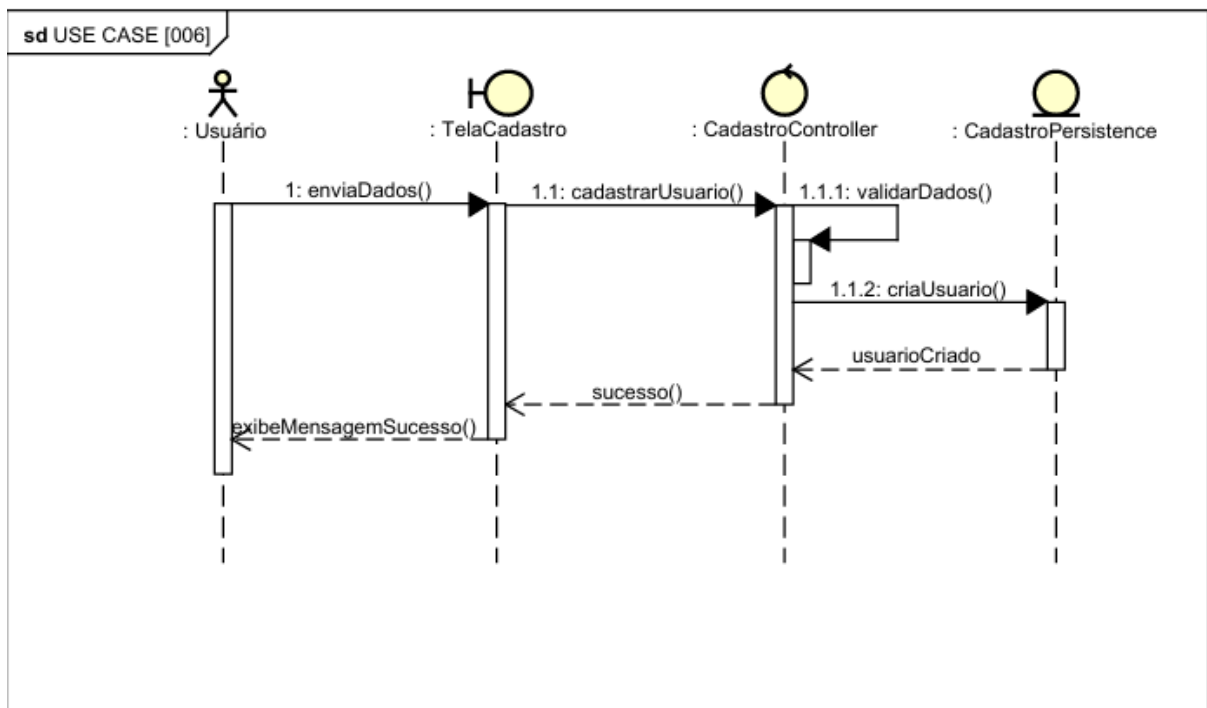
Caso de Uso [004]



Caso de Uso [005]

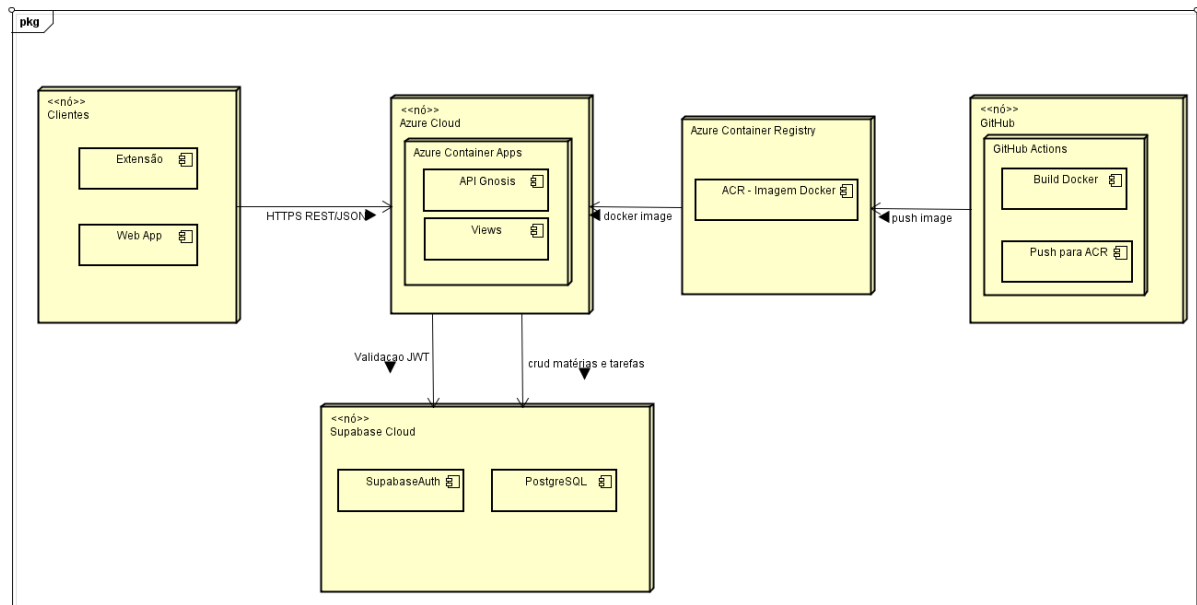


Caso de Uso [006]



7. VISÃO DE IMPLANTAÇÃO

Descrever os nodos físicos, as configurações e os artefatos que serão implantados.



8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Enumerar os itens relativos ao volume de acesso aos recursos da aplicação:

- Número estimado de usuários: 6.000
- Número estimado de acessos diários: 2.995
- Número estimado de acessos por período: **1000**
- Tempo de sessão de um usuário: 1h

8.2 Performance

Enumerar os itens referentes à resposta esperada do sistema:

- Tempo máximo de execução por transação: Até 3 segundos para a conclusão de qualquer transação síncrona padrão sob condições normais de carga.
- Tempo médio de resposta das APIs (Latência): O tempo médio de resposta para requisições de leitura (operações de consulta) deve ser inferior a 500 milissegundos em condições normais de operação.
- Percentis de consistência (Métricas P95 e P99): Pelo menos 95% das requisições globais do sistema devem ser respondidas em menos de 1 segundo.
- Pelo menos 99% das requisições devem ser respondidas em menos de 2 segundos, limitando a ocorrência de picos de lentidão para o usuário final.

- Capacidade de concorrência simultânea: O sistema deve ser capaz de suportar até 1000 usuários realizando requisições exatamente ao mesmo tempo, sem que ocorra degradação nos tempos de resposta homologados.
- Vazão do sistema (Throughput): A infraestrutura e a aplicação devem ser capazes de processar uma vazão mínima de 100 requisições por segundo (RPS) em horários de pico.
- Tempo de carregamento inicial da interface (Frontend): O tempo para o carregamento e renderização completa da tela inicial (First Contentful Paint) não deve ultrapassar 2 segundos quando acessado sob uma conexão estável de internet.
- Consumo de recursos do servidor: Sob carga normal ou de pico estimado, o consumo médio de CPU e memória RAM dos servidores de aplicação/banco de dados não deve ultrapassar 70% da capacidade total disponível, mantendo uma margem de segurança para escalabilidade.

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	Capacidade do sistema de suportar o aumento do volume de dados e do número de acessos simultâneos (como os picos estimados) sem sofrer degradação no tempo de resposta ou travamentos.	Adoção de escalonamento horizontal em ambiente de nuvem, uso de balanceadores de carga (Load Balancers) para distribuir o tráfego e implementação de paginação e indexação otimizada no banco de dados.
Confiabilidade, Disponibilidade	Garantia de que a aplicação estará operante e acessível aos usuários quando solicitada, minimizando os tempos de inatividade (<i>downtime</i>) e sendo resiliente a falhas de componentes.	Hospedagem em infraestrutura de alta disponibilidade Supabase, rotinas de backups automatizados, redundância de dados e monitoramento contínuo de <i>health checks</i> .
Portabilidade	Facilidade com que o software (código-fonte e banco de dados) pode ser transferido de um ambiente de execução para outro (de desenvolvimento para homologação/produção, ou entre diferentes provedores) sem	Utilização de containerização (ex: Docker) para isolar a aplicação e suas dependências, separação clara das variáveis de ambiente e adoção de padrões web universais (APIs RESTful).

	necessidade de reescrita.	
Segurança	Grau de proteção da aplicação contra acessos não autorizados, ataques cibernéticos e vazamentos de dados, garantindo a integridade, confidencialidade e privacidade das informações dos usuários.	Implementação de autenticação via tokens JWT, tráfego de dados protegido por criptografia ponta a ponta (HTTPS/TLS), armazenamento de senhas com <i>hash</i> e validação/sanitização estrita de entradas no <i>backend</i> para prevenir injeções de código.